

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО  
Факультет Программной инженерии и компьютерной техники

Нейроинформатика  
Лабораторная работа №2

Выполнил студент  
Егорова Варвара Александровна  
Группа Р3323  
Преподаватель: Русак Алена Викторовна

## Содержание

|                              |   |
|------------------------------|---|
| Задание.....                 | 3 |
| Ход работы.....              | 4 |
| Ход эксперимента.....        | 4 |
| Результаты эксперимента..... | 8 |
| Исходный код.....            | 9 |
| Вывод.....                   | 9 |

**Задание**

Провести классификацию объектов на изображениях с использованием сверточных нейронных сетей.

1. Для заданного набора данных построить и обучить базовую модель, состоящую из одного сверточного слоя и одного полносвязного слоя. Оценить качество работы модели.

2. Провести эксперименты по улучшению модели, изменяя параметры сверточного слоя и добавляя новые слои.
3. Подготовить выводы о влиянии параметров на качество работы сверточной нейронной сети.

### 3

#### **Ход работы**

Для работы использовался датасет "Animals-10" с Kaggle, содержащий изображения животных. В качестве архитектуры модели использовалась сверточная нейронная сеть (CNN) с функцией активации ReLU на скрытых слоях и Softmax на выходном слое для многоклассовой

классификации.

### Ход эксперимента

Все изображения в датасете были приведены к единому размеру 128x128 пикселей. Значения пикселей были нормализованы (делением на 255) для приведения их к диапазону [0, 1]. Датасет был разделен на обучающую и валидационную выборки в пропорции 80/20. Ниже приведён листинг с полной обработкой датасета.

Листинг 1 - Обработка датасета.

```
import kagglehub
import os

path = kagglehub.dataset_download("alessiocrrado99/animals10")
data_dir = os.path.join(path, "raw-img")

folders = [f for f in os.listdir(data_dir) if
os.path.isdir(os.path.join(data_dir, f))]
print("Папки классов на диске:", folders)
print("Количество классов:", len(folders))

from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_gen = datagen.flow_from_directory(
    data_dir,
    target_size=(128,128),
    color_mode='rgb',
    batch_size=32,
    class_mode='categorical',
    subset='training',
    classes=classes_list
)

4

val_gen = datagen.flow_from_directory(
    data_dir,
    target_size=(128,128),
    color_mode='rgb',
    batch_size=32,
```

```

class_mode='categorical',
subset='validation',
classes=classes_list
)

```

Для решения задачи классификации были последовательно разработаны и обучены три модели сверточной нейронной сети (CNN) с разной степенью сложности. Все модели компилировались с оптимизатором Adam, функцией потерь categorical\_crossentropy и метрикой accuracy.

Целью создания первой модели было построение базовой архитектуры для оценки исходного качества классификации. Архитектура была намеренно простой и состояла из:

- входного слоя;
- одного сверточного слоя с 8 фильтрами;
- слоя подвыборки с окном 2x2 для уменьшения размерности карты признаков;
- слоя выравнивания для преобразования двумерной карты признаков в одномерный вектор;
- выходного полносвязного слоя с 10 нейронами (по числу классов).

Ниже представлен листинг реализации модели.

Листинг 2 - Реализация базовой модели.

```

from tensorflow import keras

model = keras.models.Sequential([
    keras.layers.InputLayer(shape=(128, 128, 3)),
    keras.layers.Conv2D(filters=8, kernel_size=3, strides=1,
                        padding='valid', activation='relu',
                        use_bias=False),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=train_gen.num_classes,
                        activation='softmax', use_bias=False)
])

model.compile(optimizer='adam',

```

5

```

loss='categorical_crossentropy',
metrics=['accuracy'])

```

Вторая модель была разработана для улучшения качества

классификации за счет углубления сети и добавления механизма регуляризации для борьбы с переобучением. Она состоит из: ● входного слоя;

- первого сверточного слоя с 32 фильтрами;
- слоя подвыборки 2x2;
- второго сверточного слоя с 64 фильтрами;
- слоя подвыборки 2x2;
- слоя выравнивания;
- слоя регуляризации с коэффициентом 0.5 для случайного отключения 50% нейронов на этапе обучения;
- выходного полносвязного слоя с 10 нейронами.

Ниже представлен листинг реализации модели.

Листинг 3 - реализация улучшенной модели.

```
model2 = keras.Sequential([
    keras.layers.InputLayer(shape = (128, 128, 3)),
    keras.layers.Conv2D(filters = 32,
                        kernel_size = (3, 3),
                        activation = 'relu',
                        use_bias = True),
    keras.layers.MaxPooling2D(pool_size = (2, 2)),
    keras.layers.Conv2D(filters = 64,
                        kernel_size = (3, 3),
                        activation = 'relu',
                        use_bias = True),
    keras.layers.MaxPooling2D(pool_size = (2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dropout(rate = 0.5),
    keras.layers.Dense(10, activation = 'softmax'),
])
model2.compile(
    optimizer = 'adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)
```

Третья модель представляет собой наиболее глубокую архитектуру, разработанную для достижения максимальной точности. В её архитектуру

входят:

- входной слой;

- первый блок (сверточный слой с 32 фильтрами 3x3 и слой подвыборки 2x2);
- второй блок (сверточный слой с 64 фильтрами 3x3 и слой подвыборки 2x2);
- третий блок (сверточный слой с 128 фильтрами 3x3 и слой подвыборки 2x2);
- слой выравнивания;
- слой регуляризации с коэффициентом 0.5;
- дополнительный полносвязный слой с 128 нейронами и активацией;
- выходной полносвязный слой с 10 нейронами.

Ниже представлен листинг реализации модели.

Листинг 4 - Реализация улучшенной модели с 3 сверточными слоями.

```
model3 = keras.Sequential([
    keras.layers.InputLayer(shape=(128, 128, 3)),

    keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Flatten(),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model3.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```



Ниже в таблицах и на рисунках приведены результаты обучения и проверки моделей на собственных картинках, а также сравнение метрик.

| Реальный класс изображения | Предсказанный класс | Уверенность модели |
|----------------------------|---------------------|--------------------|
| dog                        | squirrel            | 0.577              |
| squirrel                   | cat                 | 0.863              |
| spider                     | spider              | 0.980              |
| elefant                    | elefant             | 0.979              |
| squirrel                   | dog                 | 0.582              |
| sheep                      | cow                 | 0.510              |

Таблица 1 - Результаты работы первой модели.

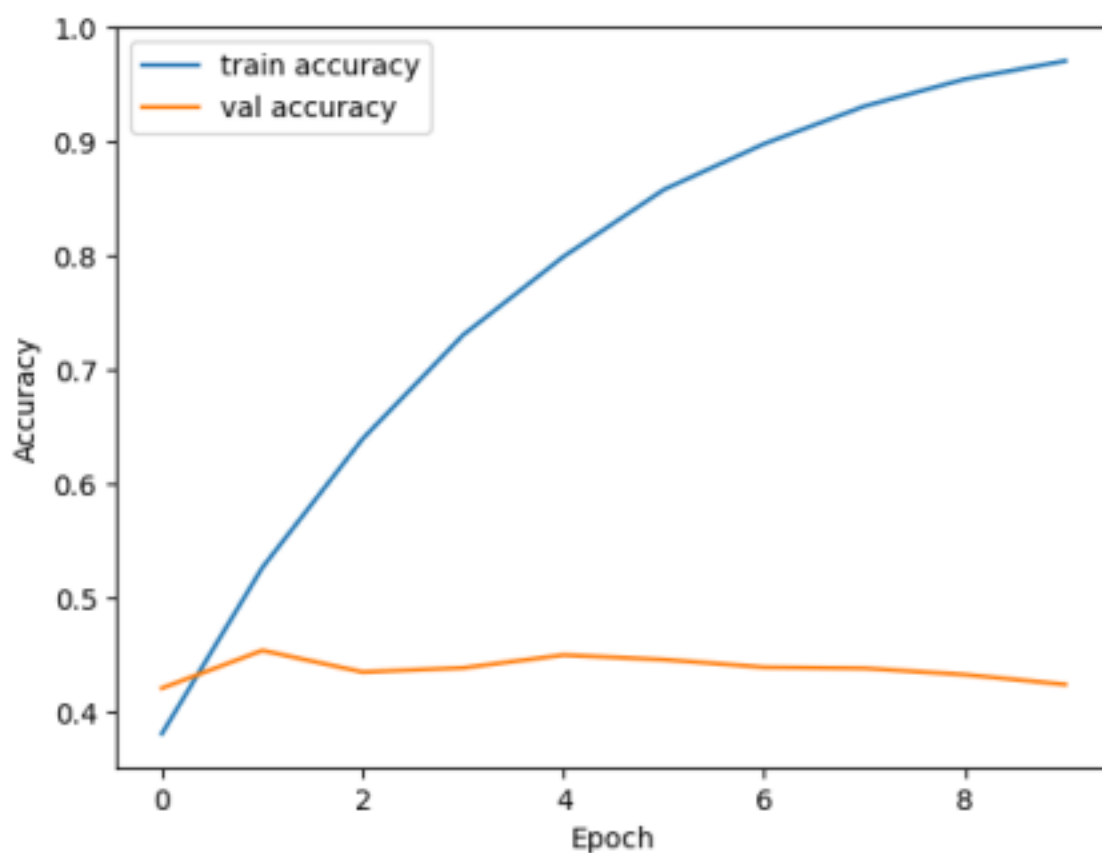


Рисунок 1 - Точность первой модели.

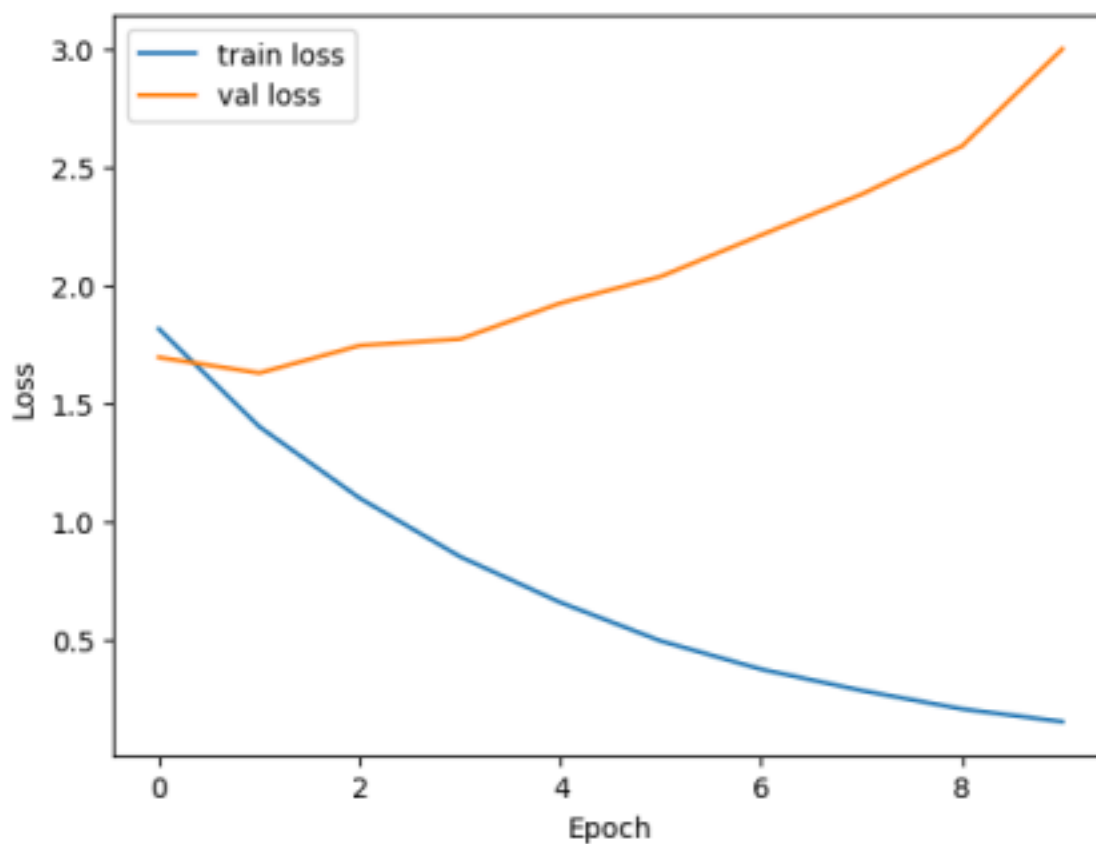


Рисунок 2 - Loss первой модели.

| Реальный класс изображения | Предсказанный класс | Уверенность модели |
|----------------------------|---------------------|--------------------|
| dog                        | dog                 | 0.970              |
| squirrel                   | squirrel            | 0.954              |
| spider                     | spider              | 0.995              |
| elefant                    | elefant             | 0.989              |
| squirrel                   | dog                 | 0.519              |

sheep sheep 0.962

Таблица 2 - Результаты работы второй модели.

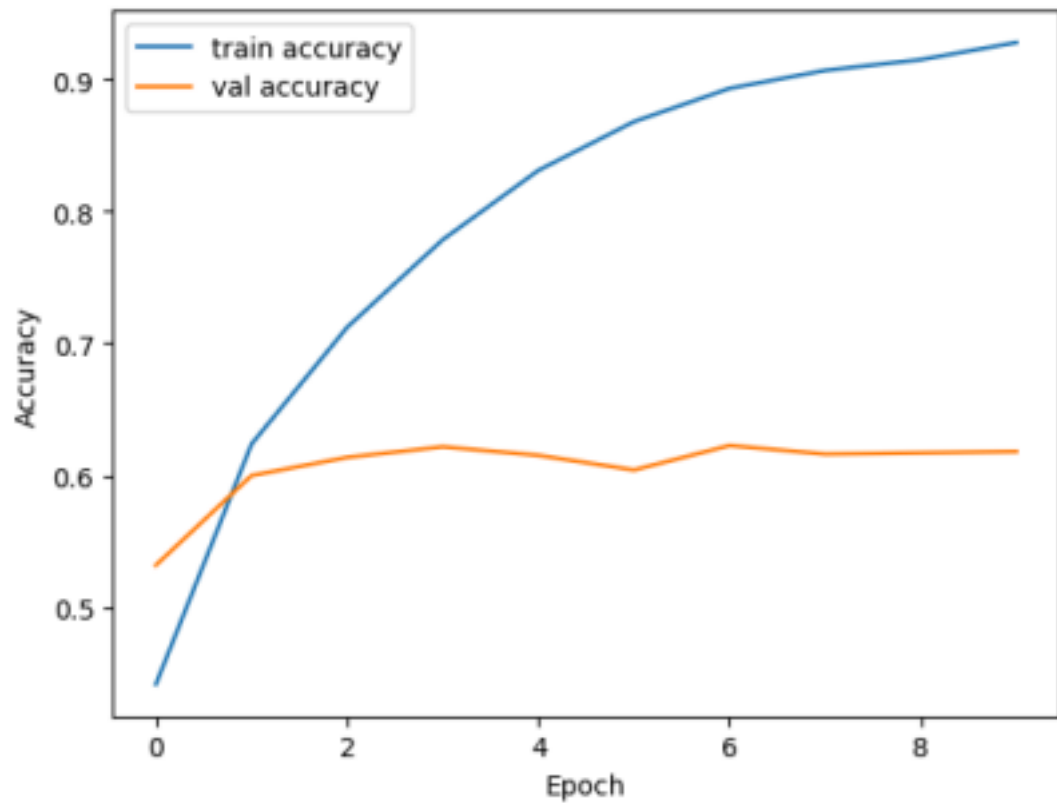


Рисунок 3 - Точность второй модели.

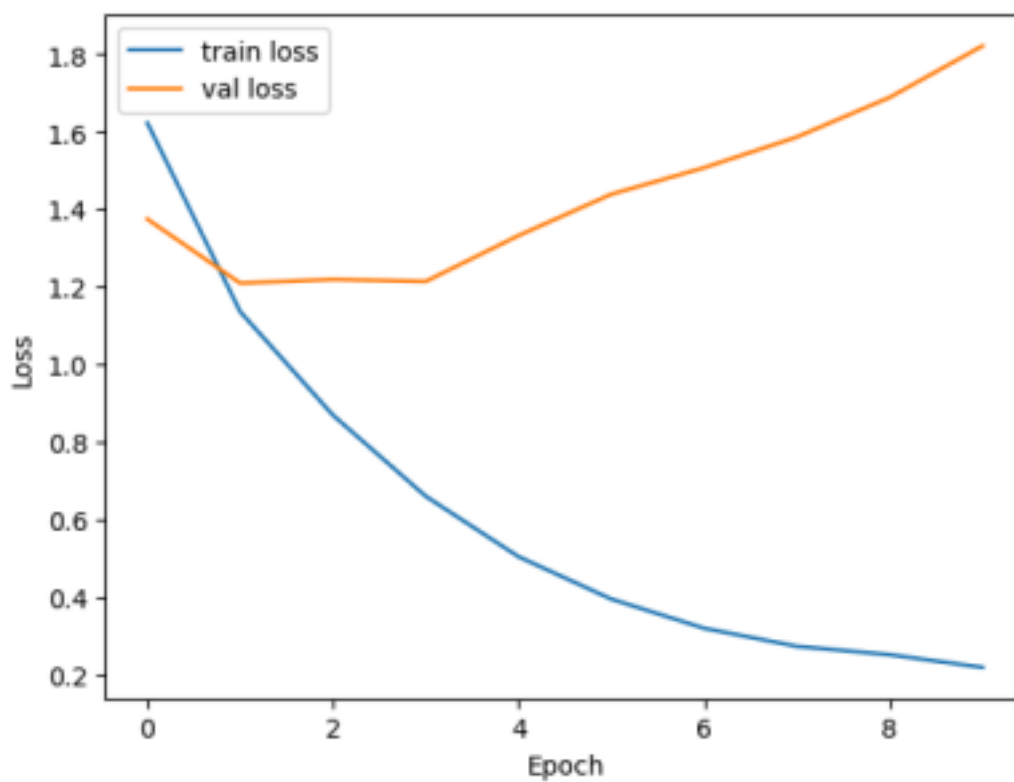


Рисунок 4 - Loss второй модели.

10

| Реальный класс изображения | Предсказанный класс | Уверенность модели |
|----------------------------|---------------------|--------------------|
| dog                        | dog                 | 0.392              |
| squirrel                   | squirrel            | 0.979              |
| spider                     | spider              | 0.919              |
| elefant                    | elefant             | 0.983              |
| squirrel                   | squirrel            | 0.745              |
| sheep                      | sheep               | 0.887              |

Таблица 3 - Результаты работы третьей модели.

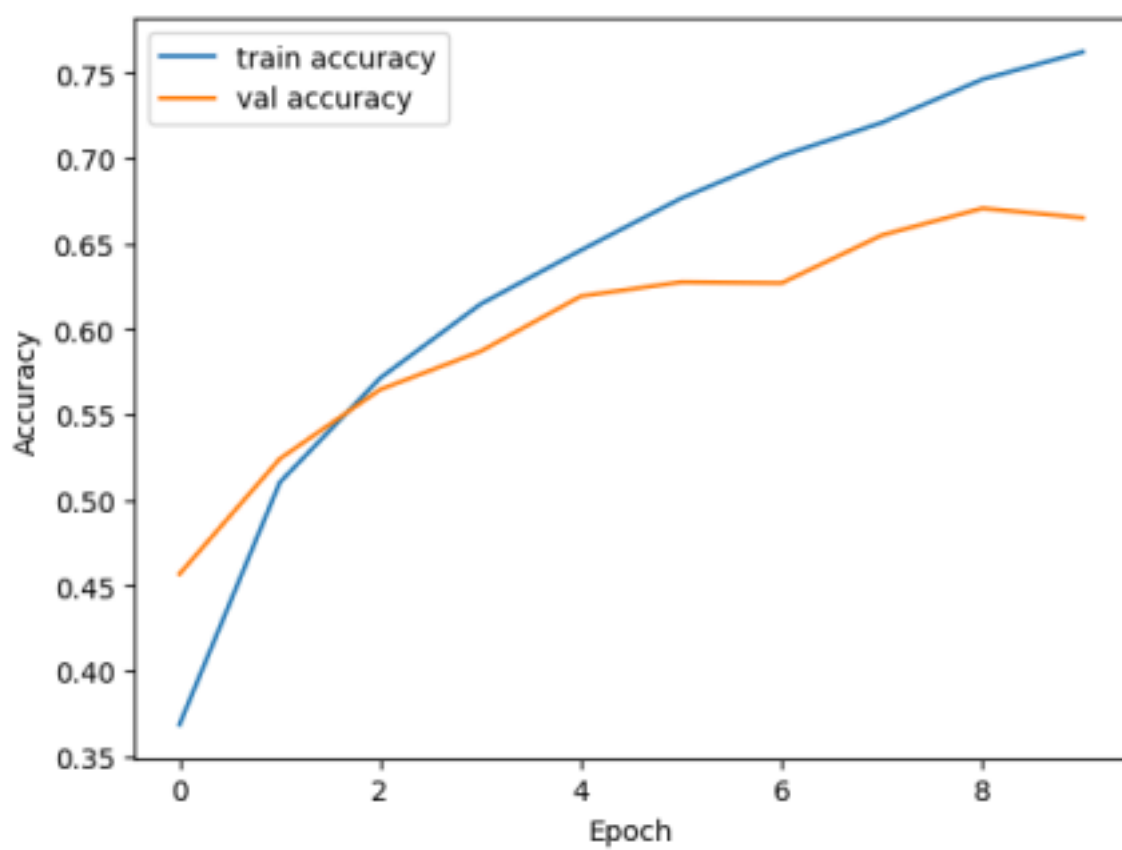


Рисунок 5 - Точность третьей модели.

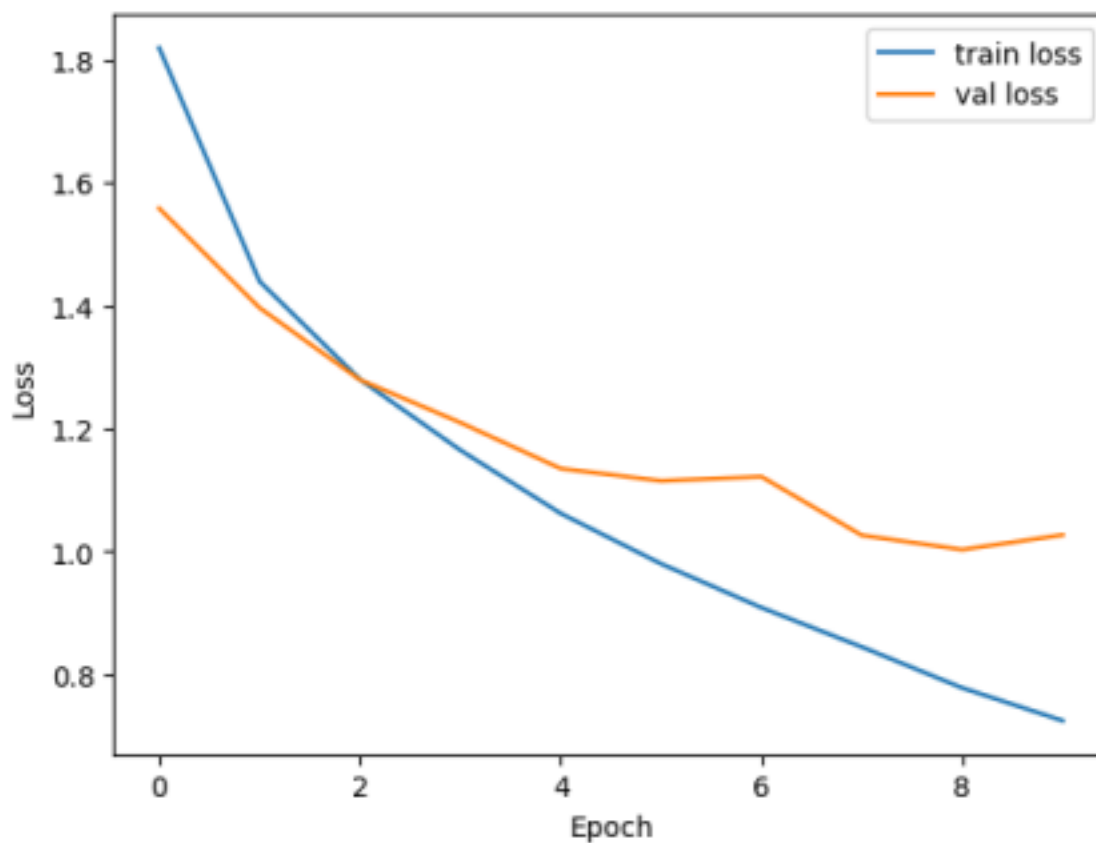


Рисунок 6 - Loss третьей модели.

| Модель | Точность на валидации |
|--------|-----------------------|
| model  | 0.451                 |
| model2 | 0.606                 |
| model3 | 0.687                 |

Таблица 4 - Сравнение точностей 3 моделей.

#### Исходный код

Исходный полный код программы представлен в Google Colab по ссылке: [NeuroInf\\_lab2.ipynb](#)

**Вывод**

В ходе работы были успешно построены и обучены три модели сверточной нейронной сети для классификации изображений. Экспериментально подтверждено, что усложнение архитектуры сети и увеличение глубины (добавление сверточных слоев) напрямую повышает точность классификации. В ходе экспериментов была выявлена и решена проблема сильного переобучения моделей. Для борьбы с ней была применена регуляризация с помощью слоя Dropout. Таким образом, для построения эффективной модели классификации необходим комплексный подход, включающий правильный выбор архитектуры, качественную подготовку данных и применение методов регуляризации.

